Can a computer that listens find songs my friends like?

A song taste engine with CLAP audio embeddings

Adiya

FRP 2026, AI Engineering track, ISSAI, Nazarbayev University

October 2026

Taste Engine is a small music recommender that compares songs by their sound, not by their genre label. It turns a 30-second clip of each song into 512 numbers with the open model CLAP, and recommends the five unrated songs closest to the songs I rated 4–5 stars. In a test with three friends, the engine found 6 of the 9 hidden favourite songs in its top 5; picks made at random from the same 293 unseen songs would find about 0.15. The test is small: three friends, one library of 300 songs, and one listener's taste.



Figure 1: Our idea. The computer listens to a song and turns it into a "sound fingerprint" of numbers. On a map of all songs, new songs near my favourites get a ring. (Illustration made with an image model.)

1 Introduction

I choose songs by how they sound. A playlist labelled "indie rock" can hold two songs that sound completely different, and two songs from different genres can sound almost the same. Most recommenders learn from other people's listening: playlists, likes, skips (collaborative filtering). That works well for big services, but it cannot work when a song or a listener is new and has no history. Content-based recommenders use the song itself instead, which fits a small library. My question: can a model that only listens to the audio find songs I would really like?
Our contributions:

- A Taste Engine web page: A page where users rate songs 1–5 stars, and *Get* 5 recommendations from a 300-song library, each with the liked song it is closest to based strictly on audio similarity;
- A collection of 376 songs stored as CLAP audio embeddings with a map that places every song by its sound;

- Three Tests: (1) A style perception test, (2) a friend test hiding a listener's 3 favourite tracks to measure how many reappear in the top, (3) a test verifying that browser embeddings match GPU.

2 Related work

Music recommenders use two main ideas [2]. *Collaborative filtering* uses what many listeners do ("people who liked X also liked Y"). It cannot help a new song or a new user with no history. *Content-based* methods use the song itself, for example its audio. Van den Oord et al. trained a neural network that predicts listening patterns from the audio, so that new songs with no listeners can be recommended too [4]. We use CLAP, a model that puts audio and text in one shared space; it was trained on more than 630,000 pairs of audio and a text that describes it [6]. To test a recommender, a common method hides some songs that a user liked and checks if they come back in the top N (here $N = 5$) [1]. Music recommendation rely on collaborative filtering (matching user patterns) and Content-based filtering (analyzing the audio itself). Filtering fails on new songs or users with no history, but content-based methods fix this. We use CLAP, a multimodal model thatlinks sound directly to text descriotions. We chose it over MERT and Essentia because they only measure pitch/tempo and only compare audio to audio.

3 The taste engine

Figure 2a shows how the engine works. It is one web page (HTML and JavaScript). The CLAP part ran as a GPU job on the class server; the page only reads the saved numbers.

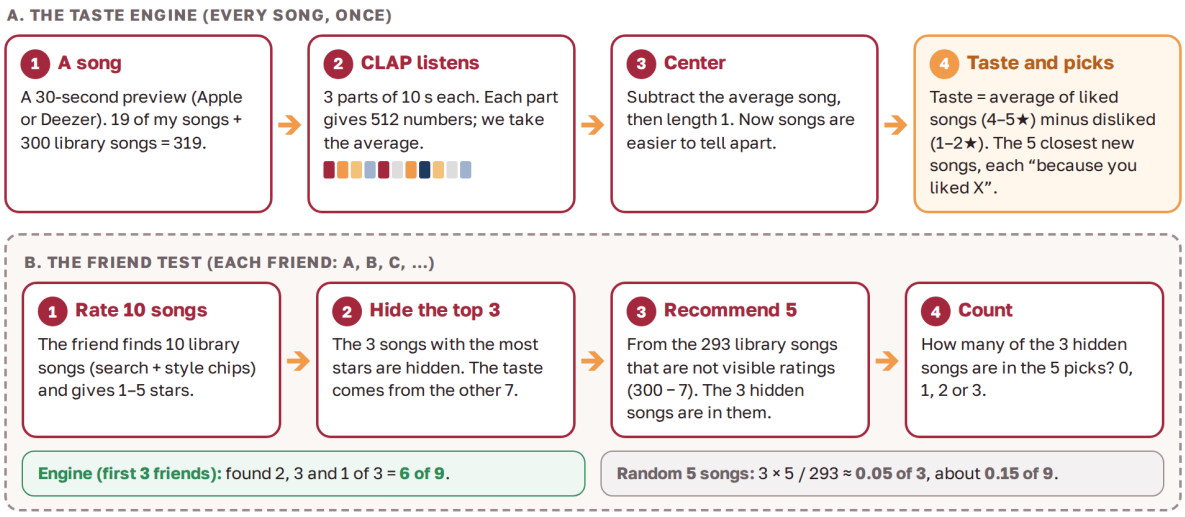


Figure 2: (a) How the taste engine works. (b) The friend test (Section 4).

3.1 The songs

My songs: I chose 76 songs that I like (Table 1). They are my taste on the page. The library. The new songs come from a library of 300 songs: 15 songs for each of 18 styles (for example jazz, k-pop, metal, city pop, lofi) and 30 songs that are popular in Kazakhstan (style "top kz"). To get the 15 songs of a style, we searched Apple for the style word (iTunes Search API [1]). So the style label is the search word, not a label from a music expert. The new picks come only from the library, never from my 76 songs. For each song we use the free 30-second preview from Apple; 60 of my songs came from Deezer and 16 from Apple. The audio is used only for the study: the page

hosts no audio files, and this paper has no audio and no lyrics. The page links to the song in Apple Music for the library songs; for the 60 songs that come from Deezer the button "Open song" opens Deezer..

Style (my label)	Songs
shoegaze	20
indie rock	20
alternative rock	16
trip pop, indie folk, pop-punk, post-punk, rock	4 each
All	**76**

Table 1: My 76 songs (my taste). The 300 library songs are the candidates for new picks.

3.2 Songs to numbers (CLAP)

We use the audio model CLAP (`laion/clap-htsat-unfused`) [6]. It turns a sound into a list of 512 numbers, an *embedding*. Songs that sound alike get embeddings that point in a similar direction. CLAP hears only 10 seconds at a time. So we cut each 30-second preview (48 kHz, mono) into 3 parts of 10 seconds, embed each part and take the average. This ran as a GPU job on one V100 (76 songs in 47 seconds). Then we *center* the embeddings: we subtract the average of all songs and make each vector length 1. The similarity of two songs is the cosine of the angle between their vectors: 1 is the same sound, smaller numbers mean a more different sound.

3.3 Taste and picks

On the page you rate songs with 1–5 stars. Your *taste vector* is the average of the songs you rated 4–5 stars minus the average of the songs you rated 1–2 stars:

$$\text{taste} = \text{mean(liked)} - \text{mean(disliked)}.$$

Every library song that you did not rate gets a score: its similarity to the taste vector. *Get 5 new songs* shows the 5 songs with the highest scores (Figure **??**a). Only the sound decides the order: the style label is shown, but it does not move a song up. The reason for each pick is the liked song that is closest to it ("because you liked Bloodhail").

If the page ranked songs by style first, the songs would be picked by labels, not by the sound. Then we would not know if the audio model works, so we rank by sound only and the test shows what does the model really hear.

3.4 The catalog map

The map shows all 376 songs as dots (Figure 3). We place them with t-SNE [5], which turns the 512 numbers of each song into 2 numbers and keeps songs that sound alike close together. The colour of a dot is the style word that found the song on Apple. An orange dot is a song you rated 4–5 stars on the page.

The orange songs are mostly in the down-left part of the map. They are close to metal, electronic, and other styles of music.

4 How we measure

4.1 A first check: 3 of my songs

Before the engine, we checked CLAP on 3 of my songs: two shoegaze songs and one indie rock song (one GPU job, embeddings not centered). Table 2 shows the similarities.

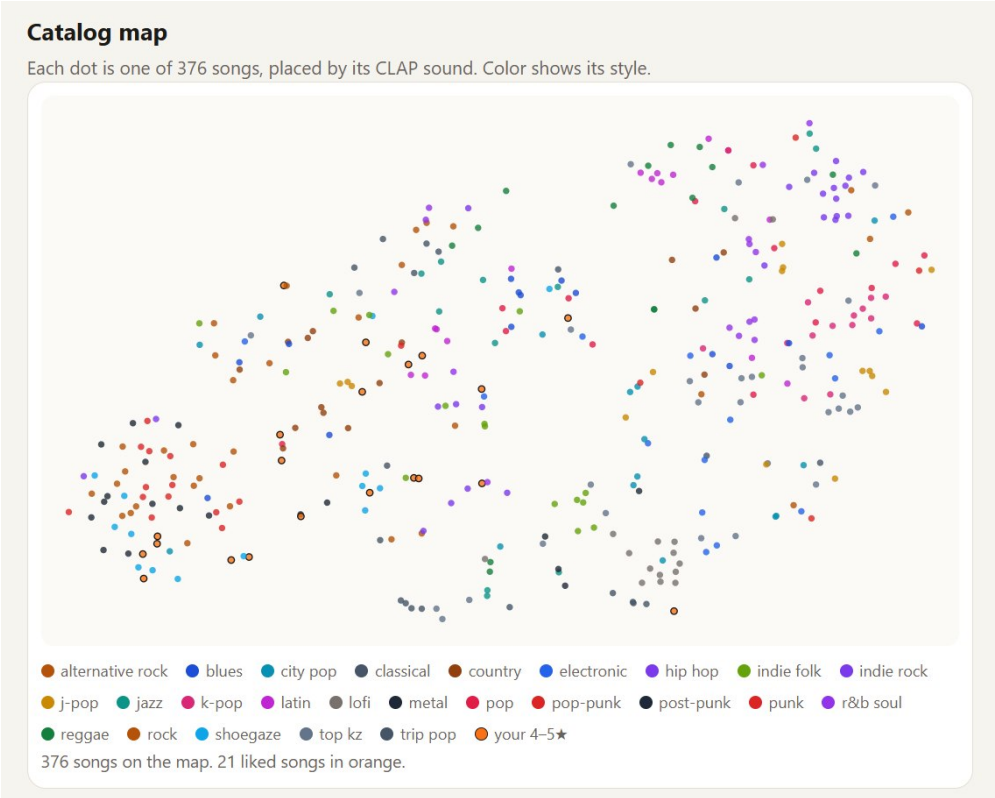


Figure 3: All 376 songs placed by their sound with t-SNE (real data from the page). Colour is the style word that found the song on Apple. An orange dot is a song you rated 4–5 stars.

4.2 Test 1: does CLAP hear the style?

For each of the 270 style songs we take the 5 songs that sound most like it (centered CLAP, cosine) and count how many have the same style word. Only the sound is used. Random picks would give the same style with the chance

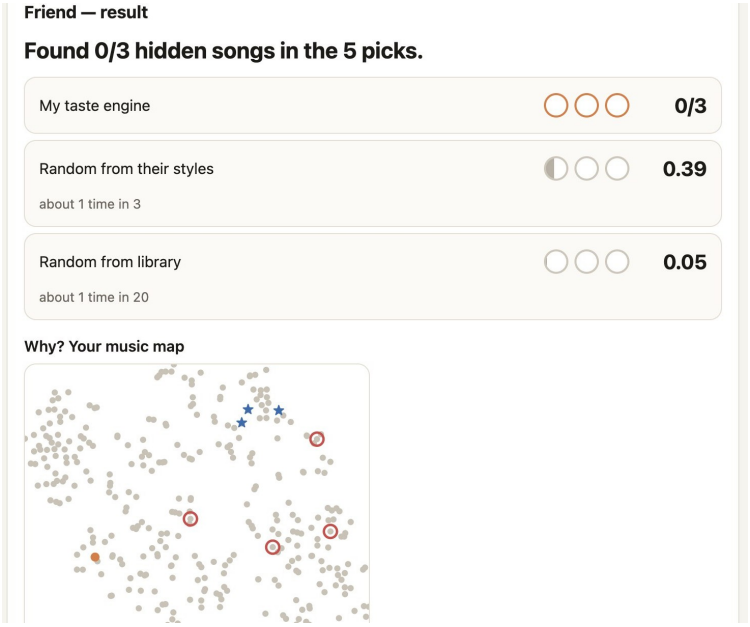
$$\frac{15-1}{270-1} = \frac{14}{269} \approx 0.052.$$

4.3 Test 2: the friend test

3 friends did the test on the page (Figure 2b). The first 3 friends did it on 27 October 2026. Each friend:

1. found 10 songs from the library with the search box and the style chips, and rated each one with 1–5 stars;
2. the page hid the 3 songs with the most stars and made the taste vector from the other 7;
3. the page picked 5 songs from the 293 library songs that are not among the 7 visible ratings ($300 - 7 = 293$; the 3 hidden songs are in them);
4. the friend sent us the result line ("found 2/3").

Found is the number of hidden songs in the 5 picks (0 to 3).



4.4 Two baselines

Random. A random recommender picks 5 of the 293 songs. Each hidden song is in these 5 with the chance 5/293. So for 3 hidden songs, random picks find on average

$$3 \times \frac{5}{293} = \frac{15}{293} \approx 0.05 \text{ of 3 songs for one friend.}$$

Same style. The friends chose their songs with style chips, so a simple rule could also work: "pick 5 random songs from the styles of the songs the friend liked". We measure this rule on the same friends.

4.5 Test 3: the same CLAP in the browser

The page can also run CLAP itself, with transformers.js [3] and a browser version of the same model (`Xenova/clap-htsat-unfused`, 34 MB). A visitor drops a song, and the page finds the 5 library songs that sound most like it. We compare the browser numbers with the GPU numbers of the same songs.

5 Results

	Bloodhail (shoegaze)	Sugar Hiccup (shoegaze)	Earth Moon Transit (indie rock)
Bloodhail	1.00	**0.92**	0.76
Sugar Hiccup	**0.92**	1.00	0.79
Earth Moon Transit	0.76	0.79	1.00

Table 2: CLAP similarity of 3 of my songs (1.00 = the same sound). The two shoegaze songs are the closest.

Test 1. Over all 270 style songs, 0.342 of the 5 nearest songs have the same style; random gives 0.052 (Figure 4). Some styles are easy for CLAP (lofi 0.71, hip hop 0.67), some are hard (pop 0.07, electronic 0.07, rock 0.13). Rock is low partly because of the search word: Apple also gave a hip hop song and a k-pop song that have "Rock" in the title. If we count rock, punk and metal as one

family, the rock songs have 0.40 family songs among their 5 nearest (random: 0.16). Table 3 shows the same thing for a rock fan.

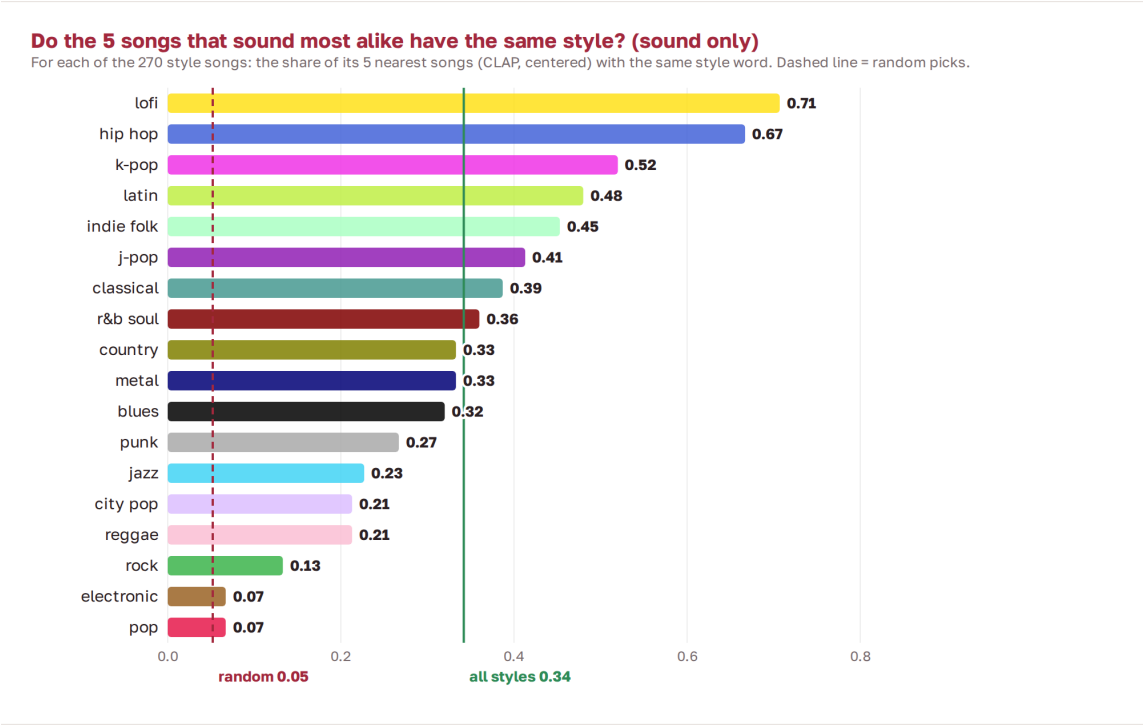


Figure 4: Test 1, sound only: for each style, the share of the 5 nearest songs with the same style word. Green line: all styles (0.34). Dashed line: random (0.05).

#	Pick (title, artist)	style word
1	The Great Escape, Boys Like Girls	punk
2	I Don't Care, Fall Out Boy	punk
3	Animal I Have Become, Three Days Grace	metal
4	Bring Me to Life, Evanescence	metal
5	Scotty Doesn't Know, Lustra	punk

Table 3: A rock fan (liked: Smells Like Teen Spirit, In the End, Numb). By sound only, the 5 picks are labelled punk and metal, not "rock", but they sound like rock with loud guitars.

Test 2. Table 4 shows the friend test.

6 Limitations

- Only three friends took the test, and each friend gave only 10 ratings.
- The friends chose the 10 songs themselves, with style chips, from a library of only 300 songs.
- The library has 300 songs, so a "hit" is easier to get than in a catalogue of millions.
- CLAP hears only the 30-second preview, 10 seconds at a time. It does not know the lyrics or the artist.
- The style-restricted version was tested by three different friends (3 of 9 found), so its number comes from a second, also small group.
- The test measures my library and my friends' taste, not a general audience.

Friend	songs rated	found in top 5	random (expected)
A	10	2 / 3	0.05 / 3
B	10	3 / 3	0.05 / 3
C	10	1 / 3	0.05 / 3
All	30	6 / 9	0.15 / 9

Table 4: The friend test. Each friend: 3 favourite songs hidden, 5 picks from 293 library songs. Random = $3 \times 5/293$ for each friend. Add a row for each extra friend; delete empty rows.

7 Ethics

The friends were volunteers. Before the test we told them what the page does and that we write only a letter (A, B, C, . . .) and their numbers, no names. The ratings stay in the friend's browser; only the result line was sent to us. The songs belong to the artists: we used the free 30-second previews only for the study. The page hosts no audio files and links to Apple Music, and this paper has no audio and no lyrics. Figure 1 was made with an image model. We used an AI coding assistant (OpenCode) to write the code; we checked all numbers ourselves.

- Ratings stay in the browser; nothing is saved on a server and no account is needed.
- The friend test uses no real names: friends appear as A, B and C.
- Songs come from public 30-second previews; no listening data of other people is copied.
- CLAP is used under its Apache-2.0 licence; the model runs in a short job, not on a phone.
- The page says plainly how the test works, so a friend can see why a song was picked.

8 Conclusion

A model that only listens to the sound found 6 of 9 hidden favourites, against about 0.15 expected from random picks. When the picks stayed inside each friend's styles, it found 3 of 9, against 1.77 expected from random picks in those styles. So the sound model beats random, and it also beats random picks inside the same styles; the next step is to mix sound with style, and to repeat the test with more people so the numbers become proof instead of a hint.

Acknowledgments

We thank our mentors at ISSAI and the FRP 2026 program.

Our page: https://kitan-a.com/frp/pub/s03/taste-engine/.

References

[1] Paolo Cremonesi, Yehuda Koren, and Roberto Turrin. Performance of recommender algorithms on top-N recommendation tasks. In *Proceedings of the Fourth ACM Conference on Recommender Systems (RecSys '10)*, pages 39–46. ACM, 2010. doi: 10.1145/1864708.1864721.

[2] Yashar Deldjoo, Markus Schedl, and Peter Knees. Content-driven music recommendation: Evolution, state of the art, and challenges. *Computer Science Review*, 51:100618, 2024. doi: 10.1016/j.cosrev.2024.100618.

[3] Hugging Face. Transformers.js: State-of-the-art machine learning for the web. https://github.com/huggingface/transformers.js, 2026. Accessed 27 October 2026.

[4] Aaron van den Oord, Sander Dieleman, and Benjamin Schrauwen. Deep content-based music recommendation. In *Advances in Neural Information Processing Sys-*

tems 26 (NIPS 2013), 2013. URL https://papers.nips.cc/paper/2013/hash/b3ba8f1bee1238a2f37603d90b58898d-Abstract.html.

[5] Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9:2579–2605, 2008. URL https://www.jmlr.org/papers/v9/vandermaaten08a.html.

[6] Yusong Wu, Ke Chen, Tianyu Zhang, Yuchen Hui, Taylor Berg-Kirkpatrick, and Shlomo Dubnov. Large-scale contrastive language-audio pretraining with feature fusion and keyword-to-caption augmentation. In *ICASSP 2023 - 2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 1–5. IEEE, 2023. doi: 10.1109/ICASSP49357.2023.10095969.